

题解

hzwer

PekingUniversity

2016 年 8 月 16 日

① 取石子 算法介绍

② 线段覆盖

③ 下棋

- 在 n 个数字当中取出 m 个, 使得总和小于 k 。这是一个比较显然的背包动态规划模型, 复杂度是 $O(kmn)$ 。

- 在 n 个数字当中取出 m 个, 使得总和小于 k 。这是一个比较显然的背包动态规划模型, 复杂度是 $O(kmn)$ 。
当然, 对于 n 较小的数据, 我们甚至可以使用 2^n 的算法暴力搜索每一种匹配方式, 然后判断是否符合条件。

- 在 n 个数字当中取出 m 个，使得总和小于 k 。这是一个比较显然的背包动态规划模型，复杂度是 $O(kmn)$ 。
当然，对于 n 较小的数据，我们甚至可以使用 2^n 的算法暴力搜索每一种匹配方式，然后判断是否符合条件。
- 同时，我们注意到 B 组数据的 k 范围比较大，如果采用背包 dp 算法时间复杂度太高，我们也开不出那么大的数组来记录状态。
但是 B 组数据的 n 太小了，采用暴力搜索反而能过。所以，我们对于不同组的数据需要不同处理。

- 对于 A 组数据，我们注意到 n 和 m 的数据范围很小，只有 10，那么直接暴力搜索或者做背包动态规划都能通过。

- 对于 A 组数据，我们注意到 n 和 m 的数据范围很小，只有 10，那么直接暴力搜索或者做背包动态规划都能通过。
- 对于 B 组数据，由于 k 的范围过大，我们只能暴力搜索。

- 对于 A 组数据，我们注意到 n 和 m 的数据范围很小，只有 10，那么直接暴力搜索或者做背包动态规划都能通过。
- 对于 B 组数据，由于 k 的范围过大，我们只能暴力搜索。
- 对于 C 组数据，由于 n 的范围过大，我们只能背包 dp。

- 对于 A 组数据，我们注意到 n 和 m 的数据范围很小，只有 10，那么直接暴力搜索或者做背包动态规划都能通过。
- 对于 B 组数据，由于 k 的范围过大，我们只能暴力搜索。
- 对于 C 组数据，由于 n 的范围过大，我们只能背包 dp。
- dp 方程

令 $f(i, j)$ 表示已经取了 i 个数字，当前总和是 j 的方案是否可行，那么 $f(i, j) = f(i, j) \text{ or } f(i-1, j-a_k)$ (a_k 是我们当前枚举到的数字)。

因为只要表示是否可行，所以 f 数组只要用 `bool` 就可以了。最后统计的时候，从 k 开始倒着枚举所有的 $f(\dots, i)$ ，找到了第一个 i 就是答案了。

- ① 取石子
- ② 线段覆盖
算法介绍
- ③ 下棋

- 我们需要在数轴上放置尽可能多的线段，那么需要按照一定的顺序在数轴上加线段，假设是从左往右添加
不妨考虑这样的问题：若我们已经取了几条线段，并且已经是最优的取法，那么左边的我们已经考虑过，如何选择右边的下一条线段，才能使结果继续最优？

- 我们需要在数轴上放置尽可能多的线段，那么需要按照一定的顺序在数轴上加线段，假设是从左往右添加
不妨考虑这样的问题：若我们已经取了几条线段，并且已经是最优的取法，那么左边的我们已经考虑过，如何选择右边的下一条线段，才能使结果继续最优？
- 应该是：选择当前的 s_i 满足条件的、 e_i 尽可能小的线段作为下一条线段。
- 假设我们当前已经取到 $e_i = k$ 了，那么我们只能在 $s_i \geq k$ 的线段中寻找，这是题目要求。
 k 越大，可以取的线段就越少。那么，我们应该尽量寻找 e_i 尽可能小的线段作为下一线段，这样加入这条线段之后再找下一条线段的时候 e_i 才会小。

- 那么做法很明显了：先把所有线段按 e_i 排个序，然后每次遇到 s_i 满足条件的就加入，由于前面的 e_i 更小的更优的线段我们已经枚举过，所以这条已经是当前我们能取到的最优的线段了。
- 对于 30% 的数据，直接 2^n 暴力枚举每一种排列是否合法。

- 那么做法很明显了：先把所有线段按 e_i 排个序，然后每次遇到 s_i 满足条件的就加入，由于前面的 e_i 更小的更优的线段我们已经枚举过，所以这条已经是当前我们能取到的最优的线段了。
- 对于 30% 的数据，直接 2^n 暴力枚举每一种排列是否合法。
- 对于 60% 的数据，需要使用上面所说的贪心方法，由于 $n \leq 1000$ 只要采用 $O(n^2)$ 的排序就可以。

- 那么做法很明显了：先把所有线段按 e_i 排个序，然后每次遇到 s_i 满足条件的就加入，由于前面的 e_i 更小的更优的线段我们已经枚举过，所以这条已经是当前我们能取到的最优的线段了。
- 对于 30% 的数据，直接 2^n 暴力枚举每一种排列是否合法。
- 对于 60% 的数据，需要使用上面所说的贪心方法，由于 $n \leq 1000$ 只要采用 $O(n^2)$ 的排序就可以。
- 对于 100% 的数据， $n \leq 50000$ ，需要使用 $O(n \log n)$ 的排序算法，如快速排序。

- ① 取石子
- ② 线段覆盖
- ③ 下棋
 算法介绍

- 首先题目的描述并不是很直接，我们不妨思考：在什么样的情况下几个炮会相互攻击？

- 首先题目的描述并不是很直接，我们不妨思考：在什么样的情况下几个炮会相互攻击？
- 如果同一行、同一列有 3 个炮，就不行。因为一个炮可以隔着一个炮打到另一个炮。3 个以上那就更不必说。

- 首先题目的描述并不是很直接，我们不妨思考：在什么样的情况下几个炮会相互攻击？
- 如果同一行、同一列有 3 个炮，就不行。因为一个炮可以隔着一个炮打到另一个炮。3 个以上那就更不必说。
- 所以，题目可以简化成这样：在 n 行 m 列的棋盘上放棋子，使得每一行每一列最多只有 2 个棋子，问有几种方案。

- 因为有行和列两个维度，我们需要找到一种顺序排列棋子，不妨先考虑行后考虑列。

- 因为有行和列两个维度，我们需要找到一种顺序排列棋子，不妨先考虑行后考虑列。
- 每一行能放 0 个、1 个、2 个棋子，我们只要保证不和前面放好的列的排列冲突就可以了。

- 因为有行和列两个维度，我们需要找到一种顺序排列棋子，不妨先考虑行后考虑列。
- 每一行能放 0 个、1 个、2 个棋子，我们只要保证不和前面放好的列的排列冲突就可以了。
- 那么怎么表示前面有几个放好的列呢？

- 因为有行和列两个维度，我们需要找到一种顺序排列棋子，不妨先考虑行后考虑列。
- 每一行能放 0 个、1 个、2 个棋子，我们只要保证不和前面放好的列的排列冲突就可以了。
- 那么怎么表示前面有几个放好的列呢？
- 列是没有顺序的，我们只要记录它到底是已经有 0 个、1 个，还是 2 个棋子。那么每一个状态就都表示出来了。

- 因为有行和列两个维度，我们需要找到一种顺序排列棋子，不妨先考虑行后考虑列。
- 每一行能放 0 个、1 个、2 个棋子，我们只要保证不和前面放好的列的排列冲突就可以了。
- 那么怎么表示前面有几个放好的列呢？
- 列是没有顺序的，我们只要记录它到底是已经有 0 个、1 个，还是 2 个棋子。那么每一个状态就都表示出来了。
- $f(i, j, k)$ 表示当前已经放了 i 行、有 j 个放 1 个棋子的列、有 k 个放 2 个棋子的列（那么放 0 个棋子的列数量当然是 $m-j-k$ ）的方案数。

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放
 - 2、一个棋子，放在原来没棋子的某一系列上

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放
 - 2、一个棋子，放在原来没棋子的某一系列上
 - 3、一个棋子，放在原来有一个棋子的某一系列上

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放
 - 2、一个棋子，放在原来没棋子的某一系列上
 - 3、一个棋子，放在原来有一个棋子的某一系列上
 - 4、两个棋子，一个在原来没棋子的列上，一个在原来一个棋子的列上

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放
 - 2、一个棋子，放在原来没棋子的某一系列上
 - 3、一个棋子，放在原来有一个棋子的某一系列上
 - 4、两个棋子，一个在原来没棋子的列上，一个在原来一个棋子的列上
 - 5、两个棋子，都放在原来没棋子的列上

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放
 - 2、一个棋子，放在原来没棋子的某一系列上
 - 3、一个棋子，放在原来有一个棋子的某一系列上
 - 4、两个棋子，一个在原来没棋子的列上，一个在原来一个棋子的列上
 - 5、两个棋子，都放在原来没棋子的列上
 - 6、两个棋子，都放在原来一个棋子的列上

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放
 - 2、一个棋子，放在原来没棋子的某一系列上
 - 3、一个棋子，放在原来有一个棋子的某一系列上
 - 4、两个棋子，一个在原来没棋子的列上，一个在原来一个棋子的列上
 - 5、两个棋子，都放在原来没棋子的列上
 - 6、两个棋子，都放在原来一个棋子的列上
- 如果第 i 行放了 0 个，那么 $f(i, j, k)$ 可以从 $f(i-1, j, k)$ 转移而来。

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放
 - 2、一个棋子，放在原来没棋子的某一系列上
 - 3、一个棋子，放在原来有一个棋子的某一系列上
 - 4、两个棋子，一个在原来没棋子的列上，一个在原来一个棋子的列上
 - 5、两个棋子，都放在原来没棋子的列上
 - 6、两个棋子，都放在原来一个棋子的列上
- 如果第 i 行放了 0 个，那么 $f(i, j, k)$ 可以从 $f(i-1, j, k)$ 转移而来。
- 如果第 i 行放了 1 个，那么 $f(i, j, k)$ 可以从 $f(i-1, j+1, k-1)$ 或者 $f(i-1, j-1, k)$ 转移而来

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放
 - 2、一个棋子, 放在原来没棋子的某一系列上
 - 3、一个棋子, 放在原来有一个棋子的某一系列上
 - 4、两个棋子, 一个在原来没棋子的列上, 一个在原来一个棋子的列上
 - 5、两个棋子, 都放在原来没棋子的列上
 - 6、两个棋子, 都放在原来一个棋子的列上
- 如果第 i 行放了 0 个, 那么 $f(i, j, k)$ 可以从 $f(i-1, j, k)$ 转移而来。
- 如果第 i 行放了 1 个, 那么 $f(i, j, k)$ 可以从 $f(i-1, j+1, k-1)$ 或者 $f(i-1, j-1, k)$ 转移而来
- 如果第 i 行放了 2 个, 那么 $f(i, j, k)$ 可以从 $f(i-1, j, k-1)$ 或者 $f(i-1, j-2, k)$ 或者 $f(i-1, j+2, k-2)$ 转移而来。

- 然后我们要考虑怎么在状态之间转移答案有这么 6 种情况:
 - 1、不放
 - 2、一个棋子，放在原来没棋子的某一系列上
 - 3、一个棋子，放在原来有一个棋子的某一系列上
 - 4、两个棋子，一个在原来没棋子的列上，一个在原来一个棋子的列上
 - 5、两个棋子，都放在原来没棋子的列上
 - 6、两个棋子，都放在原来一个棋子的列上
- 如果第 i 行放了 0 个，那么 $f(i, j, k)$ 可以从 $f(i-1, j, k)$ 转移而来。
- 如果第 i 行放了 1 个，那么 $f(i, j, k)$ 可以从 $f(i-1, j+1, k-1)$ 或者 $f(i-1, j-1, k)$ 转移而来
- 如果第 i 行放了 2 个，那么 $f(i, j, k)$ 可以从 $f(i-1, j, k-1)$ 或者 $f(i-1, j-2, k)$ 或者 $f(i-1, j+2, k-2)$ 转移而来。